

TECNOLOGICO DE COSTA RICA
ESCUELA DE COMPUTACION
PRINCIPIOS DE SISTEMAS OPERATIVOS

SEGUNDO PROYECTO

Tradicionalmente, los sistemas operativos realizaban procesos que hoy en día no se consideran parte de las funcionalidades del *core* del sistema operativo. Una de ellas es el manejo de archivos indexados, conocidos como archivos **ISAM** (*Indexed Sequential Access Method*), los cuales consistían en archivos de registros junto con un archivo de índice asociado.

Sin embargo, en la actualidad es más común utilizar archivos **JSON** en formato texto legible, en lugar de archivos de registros en formato binario. Este cambio plantea nuevos desafíos y oportunidades para la manipulación eficiente de datos semiestructurados.

Objetivo del Proyecto

El objetivo de este proyecto es escribir una **librería en C** y un **programa de línea de comando** que permita:

- Indexar archivos JSON grandes.
- Recuperar porciones del archivo JSON con base en una expresión de búsqueda.

La expresión de búsqueda debe usar una sintaxis de **expresiones regulares**, de modo que sea posible recuperar todos los elementos bajo un camino (*path*) indicado, sin necesidad de parsear el JSON completo cada vez.

Estrategia de Indexación

La estrategia a utilizar consiste en:

1. Analizar el archivo JSON original y "aplanarlo", generando una lista de todas las rutas (*paths*) de los elementos presentes.
2. Para cada ruta, registrar la posición en bytes donde **empieza y termina** el valor correspondiente dentro del archivo original.
3. Almacenar esta información en un **archivo de índice** con extensión **.jnx** (JSON Index).

Una vez creado el archivo de índice, cualquier búsqueda se realizará:

- Aplicando la expresión regular (usando la librería **regex.h** de Linux) sobre las rutas almacenadas en el índice.
- Recuperando directamente los fragmentos del archivo JSON original mediante las posiciones de inicio y fin, mediante el método **fseek** de Unix.
- Retornando un arreglo JSON si el resultado de la búsqueda son múltiples elementos.

Ejemplo Ilustrativo

A continuación se muestra un ejemplo concreto del proceso de aplanado e indexación.

```
{
  "usuarios": [
    {
      "id": 1,
      "nombre": "Ana",
      "activo": true
    },
    {
      "id": 2,
      "nombre": "Luis",
      "activo": false
    }
  ],
  "config": {
    "version": "1.0",
    "opciones": {
      "modo": "auto"
    }
  }
}
```

El proceso de aplanado produce una lista de todas las rutas absolutas (similares a JSON Pointer) junto con la ubicación de cada valor en el archivo original. Los bytes se cuentan desde el inicio del archivo (posición 0).

Ruta (<i>path</i>)	Inicio (byte)	Fin (byte)
/usuarios	12	156
/usuarios/0	21	78
/usuarios/0/id	27	28
/usuarios/0/nombre	36	41
/usuarios/0/activo	50	54
/usuarios/1	83	152
/usuarios/1/id	89	90
/usuarios/1/nombre	99	105
/usuarios/1/activo	114	119
/config	161	210
/config/version	171	176
/config/opciones	185	208
/config/opciones/modo	196	201

Este índice se almacena en el archivo `datos.jnx` (en texto plano).

Búsqueda con Expresiones Regulares

La función de búsqueda aplicará **expresiones regulares** sobre las rutas contenidas en el índice, utilizando la librería estándar **regex.h** de Linux.

Ejemplos de expresiones soportadas, todas deben empezar con \$ (que es la raíz):

Expresión de búsqueda	Coincidencias obtenidas
/usuarios/[0-9]+/nombre	/usuarios/0/nombre, /usuarios/1/nombre
/config/.*	Todo el subárbol bajo /config
./activo	Todos los elementos activo en cualquier nivel

Formato de Salida

Todos los elementos encontrados mediante la expresión regular se deben retornar como:

- Un **nuevo arreglo JSON** (si se devuelven múltiples elementos).
- Un **objeto, número, booleano o string** simple (si la búsqueda coincide exactamente con un único valor escalar).

Ejemplo de salida para la búsqueda `/usuarios/[0-9]+/nombre` sobre el archivo `datos.json`:

```
["Ana", "Luis"]
```

Entregables

1. **Librería en C** (`libjsonindex.h`) que implemente:

- Función para generar el índice `.jnx` a partir de un archivo JSON.
- Función para buscar usando expresiones regulares sobre el índice.
- Función para extraer y retornar los fragmentos del JSON original.

2. **Programa de línea de comando** (`jqindex.cpp`) que permita:

```
jqindex build datos.json          # genera datos.jnx
jqindex search datos.json "$/usuarios/[0-9]+/nombre" # realiza búsqueda
```

3. **Documentación formal:**

Se debe entregar una documentación formal (en formato pdf) que incluya todas las secciones típicas (portada, introducción, análisis del problema, solución propuesta, ejecución de pruebas, análisis de resultados y conclusiones).

Consideraciones Técnicas

- **Extensión del índice:** El formato del archivo `.jnx` debe quedar documentado dentro del código y en la documentación.

- **Parseo de JSON:** No se permite utilizar librerías externas en el proyecto incluyendo para realizar el parseo de los archivos JSON. Ustedes deben escribir un pequeño parser (quick and dirty) que considere los casos generales de un archivo JSON, asumiendo que el archivo está bien formado.
- **Control de plagio:** No se permite el uso de cantidades significativas de código tomado de Internet, la copia entre grupos de estudiantes, o el uso de motores de IA para resolver este proyecto.